

1-Question: Why can't a struct inherit from another struct or class in C#?

A: Structs are value types (stored on the Stack) designed to be lightweight and have a fixed, predictable memory size. Inheritance allows for polymorphic behavior and dynamic resizing (adding new fields in derived types), which conflicts with the fixed-size nature of the Stack. Therefore, C# makes structs implicitly sealed (they cannot inherit or be inherited from), though they implicitly inherit from `System.ValueType`.

2-Question: How do access modifiers impact the scope and visibility of a class member?

A:

Access modifiers restrict or allow where a variable or method can be used:

private: Only visible inside the exact same class/struct.

internal: Visible anywhere within the same assembly (project).

public: Visible everywhere, even outside the project.

3-Question: Why is encapsulation critical in software design?

A: Encapsulation hides the internal, sensitive data of an object from the outside world. It prevents unauthorized code from modifying variables directly (like putting a negative value for Salary). Instead, it forces access through safe channels (Methods or Properties) where validation rules and logic can be applied.

4-Question: What are constructors in structs?

A: A constructor in a struct is a special method used to initialize the fields of the struct when an object is created. In structs, parameterized constructors must fully assign valid values to *all* fields before control is returned to the caller.

5-Question: How does overriding methods like ToString() improve code readability?

A: By default, calling ToString() on a custom object returns its fully qualified type name (e.g., Session6.Point), which is not helpful. Overriding it allows developers to provide a custom, human-readable string representation of the object's actual state (e.g., (X: 5, Y: 10)), which makes debugging and logging much easier.

6-Question: How does memory allocation differ for structs and classes in C#? A:

Structs (Value Types): Allocated sequentially on the **Stack**. Memory is freed automatically and instantly when the variable goes out of scope.

Classes (Reference Types): The actual object data is stored in the **Heap**, while only a small reference (pointer/address) is stored on the Stack. Memory in the Heap is managed and cleaned up periodically by the Garbage Collector.

6-Question: What is a copy constructor?

A: A copy constructor is a parameterized constructor that takes an object of the exact same class/struct as a parameter. Its job is to create a brand-new object by copying all the data (values) from the passed object into the new one, ensuring they share the

same data but live in different memory locations.

7-LinkedIn article about constructor and its types?

A: *Have you ever wondered how objects get their initial values in C#? The secret lies in "Constructors"!* A Constructor is a special method invoked automatically when a new object is created using the new keyword. Its main job is memory allocation and initialization. Here are the 3 main types you need to know:

Default Constructor: Takes no parameters. It assigns default values (like 0 for ints, null for strings) to your object's fields.

Parameterized Constructor: Accepts parameters, allowing you to pass specific values to initialize the object exactly how you want right at creation.

Copy Constructor: Takes an existing object of the same type and copies its exact state into a brand-new object. *Mastering constructors is step one to writing robust, object-oriented code! #CSharp #DotNet #Programming #OOP*

8-Question: What is Indexer, when used, as business mention cases u have to utilize it?

A: An Indexer is a special type of property that allows an object to be accessed using array-like syntax [].

When used: It is used when a class internally acts as a collection or container of items, and you want a natural, intuitive way to retrieve elements.

Business Case: Imagine an E-commerce system with a ShoppingCart class. Instead of writing `cart.GetItemAt(0)`, an indexer allows developers to directly write `cart[0]` to retrieve the first product, making the code much cleaner and easier to read.

9-Question: Summarize keywords we have learnt last lecture.

A:

struct: Defines a lightweight value type allocated on the Stack.

class: Defines a reference type allocated on the Heap.

private: Restricts access to the current class only.

public: Allows access from any part of the code.

internal: Allows access only within the same project/assembly.

new: Allocates memory and invokes the constructor to create an object.

this: Refers to the current instance of the class/struct, often used to distinguish between class fields and method parameters.

override: Used to modify or replace the implementation of an inherited method (like ToString()).